

Section 95 Editorial — Geometric Optimization And Delaunay Structure

A. Minimum Enclosing Circle

How to find it

Randomize insertion order and rebuild only when a point violates the current circle, fixing one, two, then three boundary witnesses.

Correctness

At every nesting level the violating fixed points must lie on the new optimum boundary; one to three such points uniquely determine the minimum circle for the processed prefix.

Complexity

Expected $O(n)$ time and $O(n)$ storage.

C++

```
#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
    P operator-(P b)const {
        return {
            x-b.x,y-b.y
        }
    };
    P operator*(double k)const {
        return {
            x*k,y*k
        }
    };
};
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
```

```

}
struct L {
    P p,v;
    double angle;
}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
struct C {
    P p;
    double r;
}
;
double overlap(C a,C b) {
    double d=norm(a.p-b.p);
    if(d>=a.r+b.r)return 0;
    if(d<=abs(a.r-b.r))return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}
int tangents(C a,C b) {
    double d=norm(a.p-b.p);
    if(d<=EPS)return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r})ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}
C circle2(P a,P b) {
    P c=(a+b)*.5;
    return {
        c,norm(a-b)/2
    }
;
}
C circle3(P a,P b,P c) {
    double d=2*cross(b-a,c-a);
    double aa=dot(a,a),bb=dot(b,b),cc=dot(c,c);
    P o= {
        (aa*(b.y-c.y)+bb*(c.y-a.y)+cc*(a.y-b.y))/d,(aa*(c.x-b.x)+bb*(a.x-c.x)+cc*(b.x-a.x))/d
    }
;
    return {
        o,norm(o-a)
    }
;
}
C mec(vector<P>p) {
    mt19937 rng(947311);
    shuffle(p.begin(),p.end(),rng);
    C c {
        {
            0,0
        }
        ,-1
    }
;
    auto outside=[](P x,C c) {
        return c.r<0||norm(x-c.p)>c.r+1e-9;
    }
;
    for(int i=0;i<(int)p.size();i++)if(outside(p[i],c)) {

```

```

        c= {
            p[i],0
        }
        ;
        for(int j=0;j<i;j++)if(outside(p[j],c)) {
            c=circle2(p[i],p[j]);
            for(int k=0;k<j;k++)if(outside(p[k],c))c=circle3(p[i],p[j],p[k]);
        }
    }
    return c;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;
    vector<P>p(n);
    for(auto&x:p)cin>>x.x>>x.y;
    C=c(mec(p));
    cout<<c.p.x<<' '<<c.p.y<<' '<<c.r<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def norm(a):
    return math.hypot(a[0], a[1])

def circle2(a, b):
    c = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
    return (c, norm(sub(a, b)) / 2)

def circle3(a, b, c):
    d = 2 * cross(sub(b, a), sub(c, a))
    ux = (dot(a, a) * (b[1] - c[1]) + dot(b, b) * (c[1] - a[1]) + dot(c, c) * (a[1] - b[1])) / d
    uy = (dot(a, a) * (c[0] - b[0]) + dot(b, b) * (a[0] - c[0]) + dot(c, c) * (b[0] - a[0])) / d
    center = (ux, uy)
    return (center, norm(sub(center, a)))

def minimum_circle(points):
    p = points[:]
    random.Random(947311).shuffle(p)
    circle = ((0.0, 0.0), -1.0)

    def outside(x, c):
        return c[1] < 0 or norm(sub(x, c[0])) > c[1] + 1e-09
    for i in range(len(p)):
        if outside(p[i], circle):
            circle = (p[i], 0.0)

```

```

        for j in range(i):
            if outside(p[j], circle):
                circle = circle2(p[i], p[j])
                for k in range(j):
                    if outside(p[k], circle):
                        circle = circle3(p[i], p[j], p[k])
    return circle

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    p = [tuple(d[1 + 2 * i:3 + 2 * i]) for i in range(n)]
    c, r = minimum_circle(p)
    print(f'{c[0]:.10f} {c[1]:.10f} {r:.10f}')
if __name__ == '__main__':
    main()

```

B. Delaunay Radius Sum

How to find it

Use Bowyer–Watson cavity deletion and sum the circumradii of final non-supertriangle faces.

Correctness

The cavity boundary is retriangulated with the inserted point; the empty-circumcircle invariant is restored after every insertion. Generic position makes the final triangulation unique.

Complexity

Expected $O(n^2)$ for the supplied incremental implementation.

C++

```

#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
}
P operator-(P b)const {
    return {
        x-b.x,y-b.y
    };
}
P operator*(double k)const {
    return {
        x*k,y*k
    };
}
}
;

```

```

double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
struct C {
    P p;
    double r;
}
;
double overlap(C a,C b) {
    double d=norm(a.p-b.p);
    if(d>=a.r+b.r)return 0;
    if(d<=abs(a.r-b.r))return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}
int tangents(C a,C b) {
    double d=norm(a.p-b.p);
    if(d<=EPS)return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r})ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}
C circle2(P a,P b) {
    P c=(a+b)*.5;
    return {
        c,norm(a-b)/2
    };
}
C circle3(P a,P b,P c) {
    double d=2*cross(b-a,c-a);
    double aa=dot(a,a),bb=dot(b,b),cc=dot(c,c);
    P o= {
        (aa*(b.y-c.y)+bb*(c.y-a.y)+cc*(a.y-b.y))/d,(aa*(c.x-b.x)+bb*(a.x-c.x)+cc*(b.x-a.x))/d
    };
    return {
        o,norm(o-a)
    };
}
C mec(vector<P>p) {
    mt19937 rng(947311);
    shuffle(p.begin(),p.end(),rng);
    C c {
        {
            0,0
        }
    };
}

```

```

        ,-1
    }
    ;
    auto outside=[](P x,C c) {
        return c.r<0||norm(x-c.p)>c.r+1e-9;
    }
    ;
    for(int i=0;i<(int)p.size();i++)if(outside(p[i],c)) {
        c= {
            p[i],0
        }
        ;
        for(int j=0;j<i;j++)if(outside(p[j],c)) {
            c=circle2(p[i],p[j]);
            for(int k=0;k<j;k++)if(outside(p[k],c))c=circle3(p[i],p[j],p[k]);
        }
    }
    return c;
}
struct T {
    int a,b,c;
}
;
double orient(P a,P b,P c) {
    return cross(b-a,c-a);
}
bool incircle(P a,P b,P c,P p) {
    double ax=a.x-p.x,ay=a.y-p.y,bx=b.x-p.x,by=b.y-p.y,cx=c.x-p.x,cy=c.y-p.y;
    double d=(ax*ax+ay*ay)*(bx*cy-by*cx)-(bx*bx+by*by)*(ax*cy-ay*cx)+(cx*cx+cy*cy)*(ax*by-ay*bx);
    return d>EPS;
}
pair<vector<T>,vector<pair<int,int>>>deLaunay(vector<P>p) {
    int n=p.size();
    if(n<3)return {
        {
        },
        {
        }
    }
    ;
    double minx=p[0].x,maxx=p[0].x,miny=p[0].y,maxy=p[0].y;
    for(P x:p)minx=min(minx,x.x),maxx=max(maxx,x.x),miny=min(miny,x.y),maxy=max(maxy,x.y);
    double s=max({maxx-minx,maxy-miny,1.0}),cx=(minx+maxx)/2,cy=(miny+maxy)/2;
    p.push_back({cx-10000*s,cy-8000*s});
    p.push_back({cx+10000*s,cy-8000*s});
    p.push_back({cx,cy+10000*s});
    vector<T>tri= {
        {
            n,n+1,n+2
        }
    }
    ;
    for(int x=0;x<n;x++) {
        map<pair<int,int>,pair<int,pair<int,int>>>edges;
        vector<char>bad(tri.size());
        for(int i=0;i<(int)tri.size();i++)if(incircle(p[tri[i].a],p[tri[i].b],p[tri[i].c],p[x])) {
            bad[i]=1;
            for(auto e:{pair{tri[i].a,tri[i].b},pair{tri[i].b,tri[i].c},pair{tri[i].c,tri[i].a}}) {
                auto key=minmax(e.first,e.second);
                edges[key].first++;
                edges[key].second=e;
            }
        }
    }
}

```

```

        vector<T>next;
        for(int i=0;i<(int)tri.size();i++)if(!bad[i])next.push_back(tri[i]);
        for(auto[key,data]:edges)if(data.first==1) {
            auto[u,v]=data.second;
            if(orient(p[u],p[v],p[x])<0)swap(u,v);
            next.push_back({u,v,x});
        }
        tri.swap(next);
    }
    vector<T>clean;
    set<pair<int,int>>edges;
    for(T t:tri)if(max({t.a,t.b,t.c})<n) {
        clean.push_back(t);
        edges.insert(minmax(t.a,t.b));
        edges.insert(minmax(t.b,t.c));
        edges.insert(minmax(t.c,t.a));
    }
    return {
        clean,vector<pair<int,int>>(edges.begin(),edges.end())
    }
    ;
}

double radii(vector<P>p,bool maximum) {
    auto[tri,e]=delaunay(p);
    double ans=0;
    for(T t:tri) {
        double r=circle3(p[t.a],p[t.b],p[t.c]).r;
        if(maximum)ans=max(ans,r);
        else ans+=r;
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;
    vector<P>p(n);
    for(auto&x:p)cin>>x.x>>x.y;
    cout<<radii(p,false)<<'\\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def norm(a):
    return math.hypot(a[0], a[1])

def circle3(a, b, c):

```

```

d = 2 * cross(sub(b, a), sub(c, a))
ux = (dot(a, a) * (b[1] - c[1]) + dot(b, b) * (c[1] - a[1]) + dot(c, c) * (a[1] - b[1])) / d
uy = (dot(a, a) * (c[0] - b[0]) + dot(b, b) * (a[0] - c[0]) + dot(c, c) * (b[0] - a[0])) / d
center = (ux, uy)
return (center, norm(sub(center, a)))

def orient(a, b, c):
    return cross(sub(b, a), sub(c, a))

def in_circumcircle(a, b, c, p):
    ax = a[0] - p[0]
    ay = a[1] - p[1]
    bx = b[0] - p[0]
    by = b[1] - p[1]
    cx = c[0] - p[0]
    cy = c[1] - p[1]
    det = (ax * ax + ay * ay) * (bx * cy - by * cx) - (bx * bx + by * by) * (ax * cy - ay * cx) + (cx * cx + cy * cy) * (ax * by - ay * bx)
    return det > EPS

def delaunay(points):
    n = len(points)
    if n < 3:
        return ([], [])
    minx = min((x for x, y in points))
    maxx = max((x for x, y in points))
    miny = min((y for x, y in points))
    maxy = max((y for x, y in points))
    span = max(maxx - minx, maxy - miny, 1.0)
    cx = (minx + maxx) / 2
    cy = (miny + maxy) / 2
    p = points + [(cx - 10000 * span, cy - 8000 * span), (cx + 10000 * span, cy - 8000 * span), (cx, cy + 10000 * span)]
    tri = [(n, n + 1, n + 2)]
    for idx in range(n):
        bad = [t for t in tri if in_circumcircle(p[t[0]], p[t[1]], p[t[2]], p[idx])]
        count = {}
        direct = {}
        for t in bad:
            for e in ((t[0], t[1]), (t[1], t[2]), (t[2], t[0])):
                key = tuple(sorted(e))
                count[key] = count.get(key, 0) + 1
                direct[key] = e
        gone = set(bad)
        tri = [t for t in tri if t not in gone]
        for key, cnt in count.items():
            if cnt == 1:
                u, v = direct[key]
                if orient(p[u], p[v], p[idx]) < 0:
                    u, v = (v, u)
                tri.append((u, v, idx))
    tri = [t for t in tri if max(t) < n]
    edges = set()
    for a, b, c in tri:
        edges |= {tuple(sorted((a, b))), tuple(sorted((b, c))), tuple(sorted((c, a)))}
    return (tri, sorted(edges))

def radius_sum(points, maximum=False):
    tri, _ = delaunay(points)
    values = [circle3(points[a], points[b], points[c])[1] for a, b, c in tri]
    return (max(values) if values else 0.0) if maximum else sum(values)

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])

```

```

    p = [tuple(d[1 + 2 * i:3 + 2 * i]) for i in range(n)]
    print(f'{radius_sum(p):.10f}')
if __name__ == '__main__':
    main()

```

C. Euclidean Network

How to find it

Build Delaunay edges and run Kruskal on that sparse graph.

Correctness

Every Euclidean MST edge has an empty-disk certificate and belongs to Delaunay, so discarding all other complete-graph edges preserves an optimum.

Complexity

Expected $O(n^2 + n \log n)$ here.

C++

```

#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
}
P operator-(P b)const {
    return {
        x-b.x,y-b.y
    };
}
P operator*(double k)const {
    return {
        x*k,y*k
    };
}
};
;
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
}

```

```

}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
struct C {
    P p;
    double r;
}
;
double overlap(C a,C b) {
    double d=norm(a.p-b.p);
    if(d>=a.r+b.r)return 0;
    if(d<=abs(a.r-b.r))return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}
int tangents(C a,C b) {
    double d=norm(a.p-b.p);
    if(d<=EPS)return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r})ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}
C circle2(P a,P b) {
    P c=(a+b)*.5;
    return {
        c,norm(a-b)/2
    }
;
}
C circle3(P a,P b,P c) {
    double d=2*cross(b-a,c-a);
    double aa=dot(a,a),bb=dot(b,b),cc=dot(c,c);
    P o= {
        (aa*(b.y-c.y)+bb*(c.y-a.y)+cc*(a.y-b.y))/d,(aa*(c.x-b.x)+bb*(a.x-c.x)+cc*(b.x-a.x))/d
    }
;
    return {
        o,norm(o-a)
    }
;
}
C mec(vector<P>p) {
    mt19937 rng(947311);
    shuffle(p.begin(),p.end(),rng);
    C c {
        {
            0,0
        }
        ,-1
    }
;
    auto outside=[](P x,C c) {
        return c.r<0||norm(x-c.p)>c.r+1e-9;
    }
;
    for(int i=0;i<(int)p.size();i++)if(outside(p[i],c)) {
        c= {
            p[i],0
        }
;
    }
}

```

```

        for(int j=0;j<i;j++)if(outside(p[j],c)) {
            c=circle2(p[i],p[j]);
            for(int k=0;k<j;k++)if(outside(p[k],c))c=circle3(p[i],p[j],p[k]);
        }
    }
    return c;
}
struct T {
    int a,b,c;
}
;
double orient(P a,P b,P c) {
    return cross(b-a,c-a);
}
bool incircle(P a,P b,P c,P p) {
    double ax=a.x-p.x,ay=a.y-p.y,bx=b.x-p.x,by=b.y-p.y,cx=c.x-p.x,cy=c.y-p.y;
    double d=(ax*ax+ay*ay)*(bx*cy-by*cx)-(bx*bx+by*by)*(ax*cy-ay*cx)+(cx*cx+cy*cy)*(ax*by-ay*bx);
    return d>EPS;
}
pair<vector<T>,vector<pair<int,int>>>deLaunay(vector<P>p) {
    int n=p.size();
    if(n<3)return {
        {
        }, {
        }
    }
    ;
    double minx=p[0].x,maxx=p[0].x,miny=p[0].y,maxy=p[0].y;
    for(P x:p)minx=min(minx,x.x),maxx=max(maxx,x.x),miny=min(miny,x.y),maxy=max(maxy,x.y);
    double s=max({maxx-minx,maxy-miny,1.0}),cx=(minx+maxx)/2,cy=(miny+maxy)/2;
    p.push_back({cx-10000*s,cy-8000*s});
    p.push_back({cx+10000*s,cy-8000*s});
    p.push_back({cx,cy+10000*s});
    vector<T>tri= {
        {
            n,n+1,n+2
        }
    }
    ;
    for(int x=0;x<n;x++) {
        map<pair<int,int>,pair<int,pair<int,int>>>edges;
        vector<char>bad(tri.size());
        for(int i=0;i<(int)tri.size();i++)if(incircle(p[tri[i].a],p[tri[i].b],p[tri[i].c],p[x])) {
            bad[i]=1;
            for(auto e:{pair{tri[i].a,tri[i].b},pair{tri[i].b,tri[i].c},pair{tri[i].c,tri[i].a}}) {
                auto key=minmax(e.first,e.second);
                edges[key].first++;
                edges[key].second=e;
            }
        }
        vector<T>next;
        for(int i=0;i<(int)tri.size();i++)if(!bad[i])next.push_back(tri[i]);
        for(auto[key,data]:edges)if(data.first==1) {
            auto[u,v]=data.second;
            if(orient(p[u],p[v],p[x])<0)swap(u,v);
            next.push_back({u,v,x});
        }
        tri.swap(next);
    }
    vector<T>clean;
    set<pair<int,int>>edges;
    for(T t:tri)if(max({t.a,t.b,t.c})<n) {

```

```

        clean.push_back(t);
        edges.insert(minmax(t.a,t.b));
        edges.insert(minmax(t.b,t.c));
        edges.insert(minmax(t.c,t.a));
    }
    return {
        clean,vector<pair<int,int>>(edges.begin(),edges.end())
    }
    ;
}

double radii(vector<P>p,bool maximum) {
    auto[tri,e]=delaunay(p);
    double ans=0;
    for(T t:tri) {
        double r=circle3(p[t.a],p[t.b],p[t.c]).r;
        if(maximum)ans=max(ans,r);
        else ans+=r;
    }
    return ans;
}

struct DSU {
    vector<int>p;
    DSU(int n):p(n) {
        iota(p.begin(),p.end(),0);
    }
    int f(int x) {
        return p[x]==x?x:p[x]=f(p[x]);
    }
}

;

double mst(vector<P>p) {
    int n=p.size();
    if(n<2)return 0;
    auto[tri,e]=delaunay(p);
    if(e.empty())for(int i=1;i<n;i++)e.push_back({i-1,i});
    vector<tuple<double,int,int>>all;
    for(auto[u,v]:e)all.push_back({norm(p[u]-p[v]),u,v});
    sort(all.begin(),all.end());
    DSU d(n);
    double ans=0;
    for(auto[w,u,v]:all) {
        u=d.f(u);
        v=d.f(v);
        if(u!=v)d.p[u]=v,ans+=w;
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;
    vector<P>p(n);
    for(auto&x:p)cin>>x.x>>x.y;
    cout<<mst(p)<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque

```

```

EPS = 1e-10

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def norm(a):
    return math.hypot(a[0], a[1])

def orient(a, b, c):
    return cross(sub(b, a), sub(c, a))

def in_circumcircle(a, b, c, p):
    ax = a[0] - p[0]
    ay = a[1] - p[1]
    bx = b[0] - p[0]
    by = b[1] - p[1]
    cx = c[0] - p[0]
    cy = c[1] - p[1]
    det = (ax * ax + ay * ay) * (bx * cy - by * cx) - (bx * bx + by * by) * (ax * cy - ay * cx) + (cx * cx + cy * cy) * (ax * by - ay * bx)
    return det > EPS

def delaunay(points):
    n = len(points)
    if n < 3:
        return ([], [])
    minx = min((x for x, y in points))
    maxx = max((x for x, y in points))
    miny = min((y for x, y in points))
    maxy = max((y for x, y in points))
    span = max(maxx - minx, maxy - miny, 1.0)
    cx = (minx + maxx) / 2
    cy = (miny + maxy) / 2
    p = points + [(cx - 10000 * span, cy - 8000 * span), (cx + 10000 * span, cy - 8000 * span), (cx, cy + 10000 * span)]
    tri = [(n, n + 1, n + 2)]
    for idx in range(n):
        bad = [t for t in tri if in_circumcircle(p[t[0]], p[t[1]], p[t[2]], p[idx])]
        count = {}
        direct = {}
        for t in bad:
            for e in ((t[0], t[1]), (t[1], t[2]), (t[2], t[0])):
                key = tuple(sorted(e))
                count[key] = count.get(key, 0) + 1
                direct[key] = e
        gone = set(bad)
        tri = [t for t in tri if t not in gone]
        for key, cnt in count.items():
            if cnt == 1:
                u, v = direct[key]
                if orient(p[u], p[v], p[idx]) < 0:
                    u, v = (v, u)
                tri.append((u, v, idx))
    tri = [t for t in tri if max(t) < n]
    edges = set()
    for a, b, c in tri:
        edges |= {tuple(sorted((a, b))), tuple(sorted((b, c))), tuple(sorted((c, a)))}
    return (tri, sorted(edges))

def mst(points):
    n = len(points)
    if n < 2:

```

```

        return 0.0
    _, edges = delaunay(points)
    if not edges:
        edges = [(i, i + 1) for i in range(n - 1)]
    parent = list(range(n))

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    ans = 0.0
    for w, u, v in sorted(((norm(sub(points[u], points[v])), u, v) for u, v in edges)):
        u = find(u)
        v = find(v)
        if u != v:
            parent[u] = v
            ans += w
    return ans

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    p = [tuple(d[1 + 2 * i:3 + 2 * i]) for i in range(n)]
    print(f'{mst(p):.10f}')
if __name__ == '__main__':
    main()

```

D. Largest Empty Delaunay Circle

How to find it

Enumerate the final Delaunay triangles and maximize their circumradii.

Correctness

A finite empty circle through three sites is a Delaunay face circumcircle, and every Delaunay face supplies such an empty circle.

Complexity

Expected $O(n^2)$ here.

C++

```

#include <bits/stdc++.h>
using namespace std;
const double EPS=1e-10,PI=acos(-1.0);
struct P {
    double x,y;
    P operator+(P b)const {
        return {
            x+b.x,y+b.y
        }
    };
}
P operator-(P b)const {
    return {
        x-b.x,y-b.y
    }
}

```

```

    ;
}
P operator*(double k) const {
    return {
        x*k,y*k
    }
}
;
}
;
double cross(P a,P b) {
    return a.x*b.y-a.y*b.x;
}
double dot(P a,P b) {
    return a.x*b.x+a.y*b.y;
}
double norm(P a) {
    return hypot(a.x,a.y);
}
struct L {
    P p,v;
    double angle;
}
;
P meet(L a,L b) {
    double t=cross(b.p-a.p,b.v)/cross(a.v,b.v);
    return a.p+a.v*t;
}
struct C {
    P p;
    double r;
}
;
double overlap(C a,C b) {
    double d=norm(a.p-b.p);
    if(d>=a.r+b.r)return 0;
    if(d<=abs(a.r-b.r))return PI*min(a.r,b.r)*min(a.r,b.r);
    double x=acos(clamp((d*d+a.r*a.r-b.r*b.r)/(2*d*a.r),-1.0,1.0)),y=acos(clamp((d*d+b.r*b.r-a.r*a.r)/(2*d*b.r),-1.0,1.0));
    return a.r*a.r*x+b.r*b.r*y-d*a.r*sin(x);
}
int tangents(C a,C b) {
    double d=norm(a.p-b.p);
    if(d<=EPS)return abs(a.r-b.r)<=EPS?-1:0;
    int ans=0;
    for(double z:{abs(a.r-b.r),a.r+b.r})ans+=d>z+EPS?2:(abs(d-z)<=EPS);
    return ans;
}
C circle2(P a,P b) {
    P c=(a+b)*.5;
    return {
        c,norm(a-b)/2
    }
}
;
}
C circle3(P a,P b,P c) {
    double d=2*cross(b-a,c-a);
    double aa=dot(a,a),bb=dot(b,b),cc=dot(c,c);
    P o= {
        (aa*(b.y-c.y)+bb*(c.y-a.y)+cc*(a.y-b.y))/d,(aa*(c.x-b.x)+bb*(a.x-c.x)+cc*(b.x-a.x))/d
    }
    ;
    return {
        o,norm(o-a)
    }
}

```

```

    ;
}
C mec(vector<P>p) {
    mt19937 rng(947311);
    shuffle(p.begin(),p.end(),rng);
    C c {
        {
            0,0
        }
        , -1
    }
    ;
    auto outside=[](P x,C c) {
        return c.r<0||norm(x-c.p)>c.r+1e-9;
    }
    ;
    for(int i=0;i<(int)p.size();i++)if(outside(p[i],c)) {
        c= {
            p[i],0
        }
        ;
        for(int j=0;j<i;j++)if(outside(p[j],c)) {
            c=circle2(p[i],p[j]);
            for(int k=0;k<j;k++)if(outside(p[k],c))c=circle3(p[i],p[j],p[k]);
        }
    }
    return c;
}
struct T {
    int a,b,c;
}
;
double orient(P a,P b,P c) {
    return cross(b-a,c-a);
}
bool incircle(P a,P b,P c,P p) {
    double ax=a.x-p.x,ay=a.y-p.y,bx=b.x-p.x,by=b.y-p.y,cx=c.x-p.x,cy=c.y-p.y;
    double d=(ax*ax+ay*ay)*(bx*cy-by*cx)-(bx*bx+by*by)*(ax*cy-ay*cx)+(cx*cx+cy*cy)*(ax*by-ay*bx);
    return d>EPS;
}
pair<vector<T>,vector<pair<int,int>>> delaunay(vector<P>p) {
    int n=p.size();
    if(n<3)return {
        {
        }, {
        }
    }
    ;
    double minx=p[0].x,maxx=p[0].x,miny=p[0].y,maxy=p[0].y;
    for(P x:p)minx=min(minx,x.x),maxx=max(maxx,x.x),miny=min(miny,x.y),maxy=max(maxy,x.y);
    double s=max({maxx-minx,maxy-miny,1.0}),cx=(minx+maxx)/2,cy=(miny+maxy)/2;
    p.push_back({cx-10000*s,cy-8000*s});
    p.push_back({cx+10000*s,cy-8000*s});
    p.push_back({cx,cy+10000*s});
    vector<T>tri= {
        {
            n,n+1,n+2
        }
    }
    ;
    for(int x=0;x<n;x++) {
        map<pair<int,int>,pair<int,pair<int,int>>>edges;
        vector<char>bad(tri.size());
    }
}

```

```

        for(int i=0;i<(int)tri.size();i++)if(incircle(p[tri[i].a],p[tri[i].b],p[tri[i].c],p[x])) {
            bad[i]=1;
            for(auto e:{pair{tri[i].a,tri[i].b},pair{tri[i].b,tri[i].c},pair{tri[i].c,tri[i].a}}) {
                auto key=minmax(e.first,e.second);
                edges[key].first++;
                edges[key].second=e;
            }
        }
        vector<T>next;
        for(int i=0;i<(int)tri.size();i++)if(!bad[i])next.push_back(tri[i]);
        for(auto[key,data]:edges)if(data.first==1) {
            auto[u,v]=data.second;
            if(orient(p[u],p[v],p[x])<0)swap(u,v);
            next.push_back({u,v,x});
        }
        tri.swap(next);
    }
    vector<T>clean;
    set<pair<int,int>>edges;
    for(T t:tri)if(max({t.a,t.b,t.c})<n) {
        clean.push_back(t);
        edges.insert(minmax(t.a,t.b));
        edges.insert(minmax(t.b,t.c));
        edges.insert(minmax(t.c,t.a));
    }
    return {
        clean,vector<pair<int,int>>(edges.begin(),edges.end())
    }
    ;
}

double radii(vector<P>p,bool maximum) {
    auto[tri,e]=delaunay(p);
    double ans=0;
    for(T t:tri) {
        double r=circle3(p[t.a],p[t.b],p[t.c]).r;
        if(maximum)ans=max(ans,r);
        else ans+=r;
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout<<fixed<<setprecision(10);
    int n;
    cin>>n;
    vector<P>p(n);
    for(auto&x:p)cin>>x.x>>x.y;
    cout<<radii(p,true)<<'\n';
}

```

Python

```

import sys, math, random
from collections import deque
EPS = 1e-10

def sub(a, b):
    return (a[0] - b[0], a[1] - b[1])

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

```

```

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def norm(a):
    return math.hypot(a[0], a[1])

def circle3(a, b, c):
    d = 2 * cross(sub(b, a), sub(c, a))
    ux = (dot(a, a) * (b[1] - c[1]) + dot(b, b) * (c[1] - a[1]) + dot(c, c) * (a[1] - b[1])) / d
    uy = (dot(a, a) * (c[0] - b[0]) + dot(b, b) * (a[0] - c[0]) + dot(c, c) * (b[0] - a[0])) / d
    center = (ux, uy)
    return (center, norm(sub(center, a)))

def orient(a, b, c):
    return cross(sub(b, a), sub(c, a))

def in_circumcircle(a, b, c, p):
    ax = a[0] - p[0]
    ay = a[1] - p[1]
    bx = b[0] - p[0]
    by = b[1] - p[1]
    cx = c[0] - p[0]
    cy = c[1] - p[1]
    det = (ax * ax + ay * ay) * (bx * cy - by * cx) - (bx * bx + by * by) * (ax * cy - ay * cx) + (cx * cx + cy * cy) * (ax * by - ay * bx)
    return det > EPS

def delaunay(points):
    n = len(points)
    if n < 3:
        return ([], [])
    minx = min((x for x, y in points))
    maxx = max((x for x, y in points))
    miny = min((y for x, y in points))
    maxy = max((y for x, y in points))
    span = max(maxx - minx, maxy - miny, 1.0)
    cx = (minx + maxx) / 2
    cy = (miny + maxy) / 2
    p = points + [(cx - 10000 * span, cy - 8000 * span), (cx + 10000 * span, cy - 8000 * span), (cx, cy + 10000 * span)]
    tri = [(n, n + 1, n + 2)]
    for idx in range(n):
        bad = [t for t in tri if in_circumcircle(p[t[0]], p[t[1]], p[t[2]], p[idx])]
        count = {}
        direct = {}
        for t in bad:
            for e in ((t[0], t[1]), (t[1], t[2]), (t[2], t[0])):
                key = tuple(sorted(e))
                count[key] = count.get(key, 0) + 1
                direct[key] = e
        gone = set(bad)
        tri = [t for t in tri if t not in gone]
        for key, cnt in count.items():
            if cnt == 1:
                u, v = direct[key]
                if orient(p[u], p[v], p[idx]) < 0:
                    u, v = (v, u)
                tri.append((u, v, idx))
    tri = [t for t in tri if max(t) < n]
    edges = set()
    for a, b, c in tri:
        edges |= {tuple(sorted((a, b))), tuple(sorted((b, c))), tuple(sorted((c, a)))}
    return (tri, sorted(edges))

def radius_sum(points, maximum=False):

```

```

    tri, _ = delaunay(points)
    values = [circle3(points[a], points[b], points[c])[1] for a, b, c in tri]
    return (max(values) if values else 0.0) if maximum else sum(values)

def main():
    d = list(map(float, sys.stdin.buffer.read().split()))
    n = int(d[0])
    p = [tuple(d[1 + 2 * i:3 + 2 * i]) for i in range(n)]
    print(f'{radius_sum(p, True):.10f}')
if __name__ == '__main__':
    main()

```

Witness and robustness audit

The minimum enclosing circle is determined by at most three boundary points; the rebuild loops enumerate support sets of sizes one, two, and three. Delaunay insertion removes exactly the triangles whose circumdisks conflict with the new point, and edges occurring once form the cavity boundary. The in-circle sign must agree with triangle orientation.

On small point sets, compare the enclosing circle with all one-, two-, and three-point candidates and compare the sparse Euclidean MST with complete-graph Kruskal. Check every reported Delaunay triangle's empty circumdisk. Include duplicates if allowed, cocircular sets, grids, and nearly collinear points; random insertion order is not a replacement for a deterministic degeneracy policy.